

PR Documentation Auditor — Auditoria Automatizada de Documentação em Pull Requests

Equipe 3

[GitHub](#)

Guilherme Pereira
Alexandre Moreno
Stela Nascimento
Reilson Fonseca

30/06/2026

Sumário

Disclaimer	2
1. Introdução	3
2. Metodologia	3
3. Movimento 1 — Exposição (Alinhar Estratégia)	3
4. Movimento 2 — Composição (Desenhar a Solução)	4
5. Movimento 3 — Ensaio (Construir e Testar)	4
6. Movimento 4 — Ressonância (Medir e Aprender)	5
7. Discussões Técnicas e Estratégicas	5
8. Considerações Éticas	5
9. Lições Aprendidas e Reflexões Finais	5
10. Referências	6
11. Apêndices (opcional)	6

1. Introdução

O projeto aborda o problema do envelhecimento acelerado da documentação em projetos de software de médio e grande porte. A cada Pull Request, mudanças em funcionalidades, APIs, regras de negócio, infraestrutura ou segurança podem não ser refletidas na documentação existente. O foco da solução está nas subdisciplinas de manutenção e requisitos do SWEBOK, uma vez que a documentação é um artefato de requisitos que deve acompanhar a evolução do código.

O objetivo geral é reduzir o atrito do processo de auditoria documental utilizando Inteligência Artificial. Os objetivos específicos são: (i) extrair mudanças relevantes de um diff de PR; (ii) comparar essas mudanças com a documentação existente e detectar gaps; (iii) classificar a criticidade de cada gap, com atenção especial a segurança e compliance; (iv) gerar um relatório estruturado em Markdown acessível a PMOs; e (v) persistir o histórico das análises em MongoDB para rastreabilidade.

A abordagem com IA generativa é adequada porque a tarefa de comparar semanticamente código e documentação é intensiva em interpretação de linguagem e contexto. Uma abordagem puramente determinística por exemplo, baseada apenas em diff textual, teria dificuldade em distinguir mudanças com impacto documental de refatorações internas. A IA reduz o tempo de análise de aproximadamente 15–20 minutos manuais para cerca de 30 segundos, mantendo o custo operacional próximo de zero graças ao roteamento inteligente entre provedores gratuitos.

O trabalho foi conduzido segundo a Metodologia Sinfonia, percorrendo os quatro movimentos: Exposição (alinhar estratégia), Composição (desenhar a solução), Ensaio (construir e testar) e Ressonância (medir e aprender). Além disso, uma fase adicional chamada Interlúdio foi realizada para evoluir o MVP de v0.1.0 para v0.2.0; por decisão metodológica, seus artefatos e custos foram absorvidos no Movimento 4.

2. Metodologia

O projeto produz dois entregáveis indissociáveis: a Aplicação (o produto) e o Workflow Document (o diário de bordo do desenvolvimento assistido por IA). A aplicação evidencia o resultado técnico, enquanto o Workflow Document evidencia o aprendizado, o consumo de tokens, o esforço humano e a estimativa contrafactual. Uma sem a outra estaria incompleta, pois a disciplina avalia tanto o produto quanto o processo de construção com IA.

A Metodologia Sinfonia estruturou o trabalho em quatro movimentos. A Exposição definiu o problema, as personas, a missão e o escopo do MVP. A Composição desenhou a arquitetura com C4 Model, os registros de decisões arquiteturais e o catálogo de prompts. O Ensaio construiu a CLI, os serviços de integração com LLM, os testes automatizados e o relatório de segurança. A

Ressonância consolidou o aprendizado, a coleta de feedback interno e o Canvas de Escalabilidade.

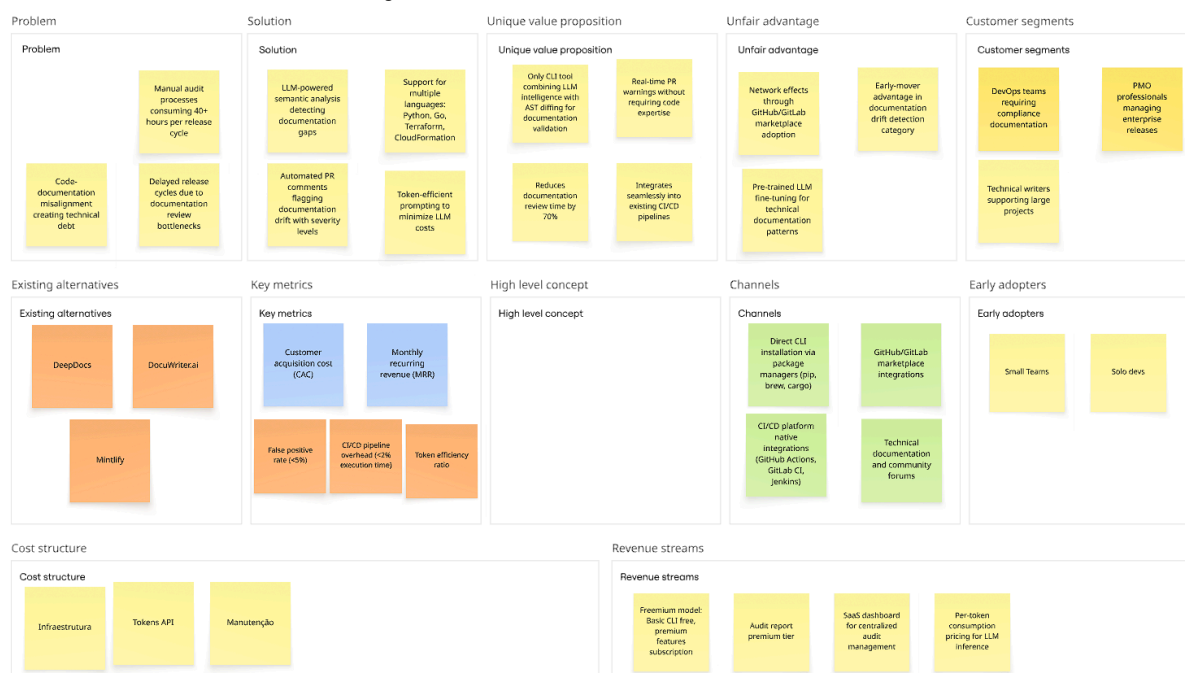
A gestão do trabalho foi organizada em sprints alinhadas aos checkpoints da disciplina: CP1 (Exposição), CP2 (Composição), CP3 (Ensaio) e Apresentação Final (Ressonância). As responsabilidades foram divididas entre desenvolvimento, liderança técnica, requisitos e QA, conforme registrado no README. O acompanhamento usou um board de tarefas documentado em `docs/TAREFAS\_CONCLUSAO\_MVP.md` e comunicação assíncrona por Discord.

O Workflow Document foi mantido como documento vivo, atualizado ao final de cada fase com o uso qualitativo de IA e as três camadas de economicidade: custo real de IA, esforço humano real e custo contrafactual humano. O documento completo entra como anexo obrigatório na Seção 12, e seu consolidado é analisado na Seção 7.

3. Movimento 1 — Exposição (Alinhar Estratégia)

O Movimento 1 — Exposição definiu a estratégia do projeto. Foram produzidos os artefatos de canvas de estratégia e ação, personas, missão/visão, métricas de sucesso, matriz de priorização e escopo do MVP.

Canvas de Estratégia e Ação



O Canvas de Estratégia e Ação consolida, em formato Lean Canvas, a tese de valor do projeto: quem sofre o problema, como a IA entra, contra quem competimos e o que torna a solução defensável.

Problema e alternativas. O gargalo central é PR entrar em produção com documentação desatualizada — auditoria manual consome 40+ horas por ciclo de release e gera dívida técnica. As alternativas existentes (DeepDocs,

DocuWriter.ai, Mintlify) atacam a geração de documentação, mas não a detecção de drift no momento do PR.

Solução e proposta de valor. Uma CLI movida a LLM que faz análise semântica do diff contra a documentação, comenta no PR com nível de criticidade e mantém custo sob controle via roteamento Groq/Gemini (ADR-005). O diferencial é ser a única ferramenta que combina inteligência de LLM com validação de drift em tempo de PR, sem exigir que o auditor (PMO) tenha expertise em código.

Clientes e early adopters. PMOs em releases com compliance e times de DevOps são o cliente primário; small teams e devs solo entram como porta de adoção pelo ciclo curto de feedback.

Canais e receita. Distribuição via package managers e integrações nativas de CI/CD; modelo freemium com tier premium (\$20/mês) e billing por token.

Personas

Foram definidas duas personas principais.

- Carolina Oliveira é PMO Sênior, com conhecimento técnico básico, responsável por garantir a qualidade e atualização da documentação antes de releases. Suas principais dores são a dependência constante de desenvolvedores, o processo manual demorado e o risco de alterações importantes passarem despercebidas. A aplicação ajuda automatizando a validação da documentação, reduzindo a dependência técnica e permitindo decisões rápidas.
- Rafael Santos é Desenvolvedor Full Stack, com conhecimento técnico avançado. Suas dores incluem esquecimento frequente de atualizar documentação, interrupções constantes durante revisões e falta de clareza sobre quais documentos alterar. A aplicação informa automaticamente o que precisa ser documentado, reduzindo retrabalho e agilizando a aprovação de PRs.

Missão e Visão

A declaração de missão do projeto é: 'Reduzir o atrito no processo de auditoria de documentação utilizando Inteligência Artificial, permitindo que equipes realizem releases com mais velocidade, menor custo operacional e maior segurança.' A visão é sustentada por cinco valores: Pragmatismo Acima da Perfeição, Eficiência com Baixo Custo, Foco no Cenário Real das Equipes, Segurança é Prioridade e Transparência nas Limitações.

O alinhamento ético é direto: a ferramenta não substitui a revisão humana, mas a auxilia; limitações são documentadas de forma transparente; e mudanças críticas de segurança nunca devem passar despercebidas, mesmo que isso signifique aceitar falsos positivos.

Métricas de Sucesso

As métricas de sucesso combinam indicadores técnicos e de negócio: tempo médio por análise inferior a 30 segundos; precisão na detecção de gaps superior a 90%; custo por análise inferior a US\$ 0,002; redução do tempo manual superior a

80%; zero falsos negativos em segurança; e clareza do relatório para o PMO sem explicação técnica adicional.

Matriz de Impacto × Esforço

A Matriz de Impacto × Esforço priorizou para o MVP as funcionalidades de alto impacto e baixo/médio esforço: extração de mudanças relevantes, detecção de gaps, classificação de criticidade, geração de relatório e persistência em MongoDB. Foram deixadas para versões futuras funcionalidades de alto esforço ou impacto ainda incerto, como dashboard web, correção automática de documentação, feedback learning e integração completa com GitHub Actions.

Escopo do MVP

O MVP é uma aplicação CLI em Node.js + TypeScript que recebe dados de um Pull Request, extrai mudanças relevantes, compara com a documentação existente, detecta gaps, classifica criticidade, gera relatório Markdown e persiste o histórico em MongoDB. As funcionalidades incluem entrada estruturada de PR, extração semântica com IA, detecção de gaps, classificação de criticidade em quatro níveis, geração de relatório e armazenamento do histórico.

Foram explicitamente deixadas fora do MVP: dashboard web, autenticação de usuários, sistema de permissões, correção automática de documentação, geração automática de PRs corrigidos, análise completa de qualidade de código, suporte robusto a múltiplas linguagens, análise de repositórios muito grandes, feedback learning automático, visualização avançada dos dados do MongoDB, integrações obrigatórias com Slack ou e-mail e a substituição da revisão humana.

4. Movimento 2 — Composição (Desenhar a Solução)

O Movimento 2 — Composição desenhou a solução. Os artefatos produzidos foram o C4 Model (níveis 1, 2 e 3), o Registro de Decisões Arquiteturais, o Catálogo de Registros de Prompt e o Canvas de Design de Experimento.

C4 Model (Níveis 1, 2 e 3)

O C4 Model descreve a arquitetura em três níveis. O Nível 1 — Contexto mostra o sistema como uma caixa central que recebe eventos do GitHub (abertura/atualização de PR), consome a API de LLM (Groq/Gemini) e entrega relatórios para PMO, desenvolvedores e segurança/compliance.

O Nível 2 — Contêineres decompõe o sistema na aplicação Node.js (CLI efêmera acionada por GitHub Actions), no MongoDB (histórico) e nas integrações com GitHub API e LLM Provider. A aplicação contém os contêineres lógicos: Entry Point, Documentation Analyzer Engine, LLM Integration Module e Report Generator + GitHub Commenter.

O Nível 3 — Componentes detalha a aplicação Node.js em seis partes: Entry Point / CLI Dispatcher, GitHubExtractorService, LLMIntegration, ReportGenerator, AnalysisRepository e Commander CLI. A integração com IA está concentrada no componente LLMIntegration, que implementa o roteamento definido no ADR-005.

Registro de Decisões Arquiteturais

Foram registradas sete decisões arquiteturais. A ADR-001 escolheu o **GITHUB_TOKEN** injetado pelo GitHub Actions em vez de PAT ou GitHub App, priorizando simplicidade e segurança.

- A ADR-002 definiu Node.js + TypeScript como stack, por startup rápido em runners efêmeros e bom ecossistema de CLI.
- A ADR-003 optou pelo MongoDB para histórico, aproveitando a flexibilidade de documentos JSON e a possibilidade de feedback learning.
- A ADR-004 decidiu pela execução via GitHub Actions (sob demanda) em vez de servidor dedicado.
- A ADR-005 é a decisão central: roteamento Groq-first com fallback para Gemini em casos sensíveis ou de grande contexto, balanceando custo, latência e segurança.
- A ADR-006 adotou Commander.js para a CLI, substituindo o parsing posicional da POC.
- A ADR-007 promoveu o extrator de PR de POC isolada em ``src/services/GitHubExtractorService.ts``, tornando-o reutilizável pelos comandos ``fetch`` e ``fetch-and-audit``.

Resumo das decisões:

ADR	Decisão	Trade-off principal
ADR-001	GITHUB_TOKEN do GitHub Actions	Token temporário, escopo automático, mas menos granular que GitHub App
ADR-002	Node.js + TypeScript CLI	Startup rápido e ecossistema CLI, mas menos libs de LLM que Python
ADR-003	MongoDB para histórico	Flexibilidade JSON e feedback learning, mas custo cloud e sem transações
ADR-004	Execução via GitHub Actions	Sem servidor, integração nativa, mas logging limitado
ADR-005	Groq-first + fallback Gemini	Custo baixo no caminho feliz e segurança nos casos críticos
ADR-006	CLI com Commander.js	Subcomandos profissionais com help/validação
ADR-007	Promoção do extrator de PR	Serviço reutilizável e testável, substituindo POC isolada

Catálogo de Registros de Prompt

O Catálogo de Registros de Prompt (versão 2) documenta os prompts centrais da solução: extração de mudanças relevantes, detecção de gaps documentais, classificação de criticidade, geração de relatório e alerta de mudanças sensíveis. Cada registro inclui objetivo, template, parâmetros, exemplos de entrada/saída e critérios de avaliação. Os prompts evoluíram da v1 para a v2 com a inclusão de structured output JSON e regras mais rígidas de segurança.

Canvas de Design de Experimento

O Canvas de Design de Experimento formula a hipótese de que uma aplicação assistida por IA consegue analisar Pull Requests, identificar mudanças relevantes, comparar com a documentação e apontar gaps de forma útil. O experimento prevê oito cenários de teste, métricas de precisão, tempo, custo e criticidade, além do critério de decisão: prosseguir com o MVP completo se os critérios mínimos forem atingidos; caso contrário, ajustar o escopo para um subconjunto de cenários.

Protótipos, wireframes ou mockups

Não foram produzidos wireframes visuais, pois a interface é uma CLI headless. O protótipo visível da solução é o próprio relatório Markdown gerado pela ferramenta, cuja estrutura e linguagem foram validadas com a persona do PMO.

5. Movimento 3 — Ensaio (Construir e Testar)

O Movimento 3 — Ensaio construiu e testou a solução. Os artefatos produzidos foram o Canvas de Testes e Validação, o relatório de segurança e o Checklist de Lançamento.

Estratégia de desenvolvimento e tecnologias

A estratégia de desenvolvimento adotou Node.js 20, TypeScript 5, Commander.js para a CLI, Vitest para testes e Mongoose para acesso ao MongoDB. O código é executado via ``tsx`` durante o desenvolvimento e compilado com ``npm run build`` para produção. As sprints foram organizadas conforme os checkpoints da disciplina: definição de escopo (CP1), arquitetura e experimentos (CP2), implementação e testes (CP3).

A implementação seguiu o princípio de Design Emergente: o Nível 3 do C4 Model foi refinado durante o desenvolvimento, e a POC do extrator de PR foi promovida a serviço oficial na fase Interlúdio. A equipe usou agentes de código (Claude, OpenCode) assistidos por workflow de spec e plano, com testes escritos antes ou junto da implementação.

Fluxo de integração com LLMs

O fluxo de integração com LLMs é comandado pelo ``LLMIntegrationService``. O roteamento definido na ADR-005 usa Groq (``llama-3.3-70b-versatile``) como caminho padrão por ser rápido e quase gratuito; quando o diff toca arquivos sensíveis (auth, ``.env``, infraestrutura, CI/CD) ou excede aproximadamente 30 mil tokens, o processamento é direcionado para o Google Gemini, que oferece maior contexto e raciocínio.

As salvaguardas incluem: structured output em JSON para reduzir variabilidade das respostas; ``parseJSONSafely`` com fallback conservador quando a resposta está malformada; retry com exponential backoff; e exclusão temporária do corpus gerado pelo comando ``fetch-and-audit``, salvo quando a flag ``--keep-corpus`` é usada.

Canvas de Testes e Validação

O Canvas de Testes e Validação cobre quatro frentes. Os testes funcionais usam Vitest, totalizando 77 testes em 12 arquivos, incluindo roteamento, parsing seguro, CRUD MongoDB e comandos CLI. Os testes de qualidade dos outputs do LLM seguem os oito cenários do Canvas de Experimento, com meta de 80% de precisão e zero falsos negativos em segurança. Os casos de borda tratam diff vazio, ObjectId inválido, diffs grandes e JSON malformado. A latência é projetada abaixo de 30 segundos, embora ainda não tenha sido medida end-to-end com APIs reais. A mitigação de alucinação combina prompts com justificativa obrigatória, structured output e regras determinísticas de roteamento.

Evidências de versionamento

O versionamento foi feito com Git, utilizando branches por frente e pull requests por feature. O repositório conta com mais de 70 commits desde a POC. O build é executado com ``npm run build`` e os testes com ``npm test``. A integração contínua ainda não foi implementada via GitHub Actions, permanecendo como item do roadmap pós-MVP.

Análise de segurança (Aula 30)

A análise de segurança da aplicação (Aula 30) está documentada em *docs/RELATORIO_SEGURANCA_APPSEC.md*. Foram identificados dez achados (AS-01 a AS-10), a maioria com severidade Alta, relacionados a mascaramento de secrets antes do envio à LLM, falha automática do job em criticidade Crítica, imutabilidade do histórico, publicação segura de relatórios, prompt injection, validação de respostas, permissões do GitHub Actions, supply chain e retenção de artefatos.

O plano de mitigação prioriza ações P0: implementar redaction de secrets, falhar o job para criticidade Crítica, tornar o MongoDB obrigatório em CI/CD e remover ou restringir operações de update/delete no histórico. Ações P1 incluem resumo sanitizado para comentários em PR, comportamento fail-closed em erros de parsing e gate de supply chain. Ações P2 incluem hash/assinatura de registros e política formal de retenção alinhada à LGPD e ISO 27001.

Priorização das ações:

Achado	Severidade	Resumo da mitigação
AS-01	Alta	Implementar redaction de secrets antes da LLM
AS-02	Alta	Falhar o job para criticidade Crítica
AS-03	Alta	Tornar MongoDB obrigatório em CI/CD

AS-04	Alta	Modelo append-only para histórico
AS-05	Alta	Publicar apenas resumo sanitizado no PR
AS-06	Média/Alta	Delimitar dados não confiáveis e regras determinísticas
AS-07	Média	Falhar fechado em erro de parsing
AS-08	Média	Permissões mínimas no GITHUB_TOKEN
AS-09	Média/Alta	Verificação de cadeia de suprimentos (npm audit)
AS-10	Média	Retenção mínima de artefatos brutos

Checklist de Lançamento

O Checklist de Lançamento considera o MVP validado para build limpo, testes passando, documentação atualizada, catálogo de prompts versionado, Workflow Document completo e relatório de segurança. Itens pendentes e honestamente declarados são: validação end-to-end com APIs reais, redaction de secrets, GitHub Actions workflow e gate de supply chain.

Evidências de funcionamento

As evidências de funcionamento incluem o exemplo de relatório Markdown documentado no README e a execução local dos testes. Prints e vídeos do terminal devem ser anexados pelo time antes da entrega final.

6. Movimento 4 — Ressonância (Medir e Aprender)

O Movimento 4 — Ressonância mediu o aprendizado e consolidou o feedback. Além dos artefatos previstos (Painel de Feedback e Insights e Canvas de Escalabilidade), este movimento absorve a fase Interlúdio, que evoluiu o produto de v0.1.0 para v0.2.0.

Validação das hipóteses

A hipótese principal de que uma aplicação assistida por IA consegue auditar documentação em Pull Requests de forma útil foi confirmada parcialmente. O fluxo central funciona end-to-end com dados simulados, mas a validação real contra APIs Groq/Gemini ainda é pendente. O pivô mais importante foi a adoção do roteamento por sensibilidade em vez de um único modelo caro, preservando custo baixo sem comprometer a segurança.

Decisão estratégica

A decisão estratégica é perseverar. O MVP demonstra que o fluxo de auditoria documental é viável com custo próximo de zero e que o roteamento inteligente

atende ao valor de segurança. As próximas iterações devem focar em: validação end-to-end com APIs reais, implementação de redaction/mascaramento de secrets, e integração como GitHub Actions.

Canvas de Escalabilidade

O Canvas de Escalabilidade divide o futuro em três eixos. O eixo técnico inclui GitHub Actions, redaction de secrets, fallback para modelos maiores e fragmentação de análise por arquivo. O eixo de produto prevê dashboard web, notificações, suporte a Swagger/Confluence/Wiki e feedback learning via histórico MongoDB. O eixo de adoção contempla publicação no GitHub Marketplace, suporte multi-linguagem e modo SaaS.

Interlúdio — evolução para v0.2.0

A fase Interlúdio conectou o MVP inicial (v0.1.0, com CLI posicional e POC separada) à versão profissional (v0.2.0). Foram implementados: quatro subcomandos via Commander.js (*audit*, *fetch*, *fetch-and-audit*, *history*), a classe *ReportGenerator* para relatórios Markdown, a promoção do extrator de PR para *GitHubExtractorService*, a persistência do histórico no MongoDB e a refatoração do entry point para um thin dispatcher.

O fluxo de spec → plano → TDD com subagentes paralelos evitou os bugs de import e build que ocorreram na fase Ensaio, resultando em 77 testes passando e build limpo. Os custos e tokens do Interlúdio foram somados aos da Ressonância nas tabelas de economicidade.

7. Economicidade do Desenvolvimento Assistido por IA

Esta seção apresenta os custos de IA em dólar (USD), pois os provedores Groq e Gemini faturam nessa moeda. Os salários e custos humanos são apresentados em real (BRL), refletindo a realidade brasileira da equipe. A análise comparativa não converte os valores pela alta volatilidade do câmbio atual, a equipe entende que a análise seria complexa e tende a se depreciar.

7.1 Camada 1 — Custo real de IA (total do projeto)

Fase	Tokens entrada	Tokens saída	Custo IA (USD)
Exposição	19.000	9.500	0,0097
Composição	81.000	39.000	0,0356
Ensaio	92.000	103.500	6,3646
Ressonância + Interlúdio	229.000	109.000	0,7260

Fase	Tokens entrada	Tokens saída	Custo IA (USD)
Total	421.000	261.000	7,1359

A Camada 1 consolidou o consumo real de tokens ao longo do projeto. A fase Ensaio concentra a maior parte do custo (US\$ 6,36), principalmente devido a uma sessão longa de pareamento entre Tech Lead e Claude Opus 4.7 para testes e hardening. Vale notar que esse custo é de desenvolvimento assistido, não do custo operacional do produto, que permanece próximo de zero com o uso de provedores gratuitos.

7.2 Camada 2 — Esforço humano real (consolidado)

A Camada 2 registra o esforço humano efetivamente gasto com assistência de IA. O total foi de 16,25 horas, sendo 6,00 horas na Composição e 5,25 horas no Ensaio. O tempo humano inclui revisão, validação e ajustes dos outputs gerados pela IA, além das decisões tomadas sem assistência.

Fase	Horas humanas com IA	Observações
Exposição	0,75	Personas e Missão, totalmente assistido por Claude
Composição	6,00	C4 Model, stack, LLM provider, ADRs, revisão crítica
Ensaio	5,25	POC, LLMIntegrationService, testes Vitest, hardening
Ressonância + Interlúdio	4,25	CRUD MongoDB, correções Windows/Mongoose, CLI v0.2.0
Total	16,25	

7.3 Camada 3 — Custo contrafactual humano (total do projeto)

Fase	Horas totais estimadas (sem IA)	Custo humano estimado (R\$)
Exposição	4,0	160,00
Composição	15,5	1.162,50
Ensaio	34,5	2.380,00
Ressonância + Interlúdio	34,0	2.060,00
Total	88,0	5.762,50

A Camada 3 estima o custo contrafactual de executar o mesmo trabalho sem nenhuma ferramenta de IA generativa. Foram usados perfis de mercado de Recife/PE em 2026: Júnior (R\$ 35–45/h), Pleno (R\$ 65–85/h), Sênior (R\$ 100–130/h) e Arquiteto (R\$ 140–180/h). O custo estimado total sem IA é de R\$ 5.762,50, correspondente a 88 horas de trabalho humano.

7.4 Análise comparativa

- Custo total com IA (R\$): R\$ 1.158,18 (tokens: R\$ 35,68 + horas humanas: R\$ 1.122,50)
- Custo total estimado sem IA (R\$): R\$ 5.762,50
- Razão de economicidade: 4,98×
- Saving estimado (R\$): R\$ 4.604,32
- Saving estimado (%): 79,9%

7.5 Limitações da análise

O contrafactual é uma estimativa subjetiva, sujeita a viés de retrospecto. Não é possível saber com precisão quanto tempo a equipe levaria sem IA, uma vez que parte do conhecimento acumulado veio do próprio uso das ferramentas.

O custo com IA não inclui a curva de aprendizado das ferramentas. Embora a equipe já tivesse alguma familiaridade, o domínio efetivo de prompts, limitações de modelos e estratégias de roteamento exigiu iterações iniciais.

Custo menor não implica qualidade equivalente em todas as dimensões. A solução ainda depende de validação end-to-end real e de controles de segurança adicionais antes de ser considerada pronta para produção.

Houve atividades em que a IA aumentou o tempo total, como a correção de um import quebrado no commit inicial, a substituição do modelo Groq descontinuado e o diagnóstico de um bug de case-sensitivity no Windows. Esses retrabalhos foram documentados explicitamente no Workflow Document.

Além disso, é difícil dizer com precisão o uso o gasto de tokens juntamente com o preço, visto que em partes do projeto foi utilizado um workflow complexo com

agentes que utilizam diferentes modelos para planejamento, codificação e revisão, o resultado é mais satisfatório, mas dificulta o tracking.

8. Discussões Técnicas e Estratégicas

As decisões arquiteturais foram justificadas pelos cinco valores do projeto: pragmatismo, baixo custo, cenário real, segurança e transparência. O roteamento Groq/Gemini (ADR-005) é o exemplo mais representativo: em vez de usar um único modelo caro, a solução usa o modelo rápido e barato no caso comum e reserva o modelo mais robusto para os casos de segurança, mantendo o custo médio baixo sem comprometer a detecção de riscos críticos.

As integrações realizadas incluem a GitHub REST API (extração de PRs, metadata e diffs), as APIs de Groq e Gemini (análise semântica) e o MongoDB via Mongoose (persistência do histórico). Nenhuma integração web ou servidor dedicado foi necessária no MVP, o que reduziu a superfície de ataque e o custo operacional.

Os principais desafios técnicos enfrentados foram: o modelo Groq *mixtral-8x7b-32768* sugerido inicialmente pela IA estava descontinuado e só foi detectado nos testes; o *tsconfig.json* original tinha *moduleResolution* implícito *classic*, impedindo o build; e um bug de comparação case-sensitive no *readDocsDirectory* só apareceu ao rodar os testes no Windows. Todos foram corrigidos com testes e validação local.

Os trade-offs centrais envolvem qualidade, custo (tokens/latência) e complexidade. Usar apenas Groq reduziria custo e latência, mas aumentaria o risco de falsos negativos em segurança. Usar apenas Gemini ou Claude aumentaria a qualidade, mas elevaria custo e tempo de resposta. A solução adotada foi o roteamento por sensibilidade, que equilibra os três fatores, aceitando a complexidade adicional de manter dois provedores.

9. Considerações Éticas

A solução manipula código de repositórios privados, diffs, relatórios e possivelmente secrets. Os principais riscos éticos identificados são: exposição de código proprietário ou dados sensíveis para provedores de LLM; viés na classificação de criticidade que possa subestimar riscos de segurança; e impacto social relacionado à substituição parcial de tarefas de revisão manual.

As estratégias de mitigação adotadas incluem: roteamento de diffs sensíveis para o provedor mais robusto; prompts que forcem justificativa e structured output; fallback conservador quando a resposta da IA é inválida; e recomendações do relatório de segurança para implementar redaction de secrets, histórico append-only e publicação apenas de resumos sanitizados.

A transparência é mantida por meio da documentação explícita das limitações no README e no relatório de segurança. O usuário final (PMO ou desenvolvedor)

permanece responsável pela decisão final de aprovação do PR; a ferramenta atua como primeiro filtro, não como árbitro.

A atribuição do uso de IA no desenvolvimento é registrada no Workflow Document e neste relatório, conforme exigido pelo Código de Conduta da disciplina. Todos os prompts, modelos utilizados, custos de tokens e lições aprendidas estão documentados de forma auditável.

10. Lições Aprendidas e Reflexões Finais

A Metodologia Sinfonia mostrou-se útil para estruturar o trabalho em etapas com artefatos claros. O Movimento 2 — Composição foi o mais valioso, pois o C4 Model e os ADRs estabeleceram uma arquitetura que sobreviveu às mudanças de implementação. O Movimento 3 — Ensaio foi o mais desafiador, exigindo caça a bugs de build e modelo descontinuado.

Avaliação da proposta de valor entregue

A proposta de valor entregue é uma CLI funcional que audita PRs em busca de gaps documentais com custo operacional próximo de zero. Embora ainda não tenha sido validada end-to-end com APIs reais, a solução demonstra o fluxo completo e fornece uma base sólida para evolução.

Pontos de melhoria

Os principais pontos de melhoria são: implementar redaction de secrets antes do envio à LLM; adicionar comportamento fail-closed para criticidade Crítica; criar o workflow de GitHub Actions; e realizar validação end-to-end com APIs reais para confirmar as projeções de latência e custo.

Aprendizados sobre o uso de IA generativa

Os aprendizados sobre o uso de IA generativa incluem: meta-prompts detalhados são a maior alavanca de produtividade; a IA não valida build por conta própria; números gerados pela IA não são equivalentes a medições reais; e o fluxo de spec → plano → TDD reduz retrabalho em comparação com geração direta de código.

Relato individual de cada integrante

<Relato individual de Guilherme Pereira >

O desenvolvimento apoiado por IA já me era um assunto familiar, durante a faculdade e trabalho já estive realizando à prática por um tempo de forma a já ter obtido certa intuição sobre seus desafios e como mitigar os problemas. Em TASI6, entretanto, consegui elevar o conhecimento para um nível além, as discussões em sala de aula de forma crítica e estimulando a reflexão fazem entender o papel que o ser humano tem em um processo produtivo que está mudando tão rápido e hoje parece ter adquirido outros paradigmas de papéis, valores e senso de produtividade. Minha contribuição para o projeto foi na parte de desenvolvimento e planejamento, além de entender que tive forte liderança na organização de equipe e apresentação de resultados parciais durante as aulas. Durante o desenvolvimento estive à frente da ideia principal e na esteira de busca e tratamento das PRs no github para uso no projeto. Meu trabalho esteve mais relacionado à código e organização e separação de tarefas para equipe. Durante

meu processo de código, utilizei bastante o claude code, onde mesclava o uso do claude sonnet com claude Opus. Posteriormente, por uma questão de custo (claude é bem caro), assinei o Opencode, utilizando modelos chineses como GLM5.1, Minimax, Deepseek e kimi-k2.6. Os modelos me entregaram resultado bastante satisfatório, bem parecido com o claude, mas com mais tokens e pela metade do preço. Considero que portanto, a disciplina me ajudou a ter um pouco mais de positividade pela área de tecnologia, me mostrando que o ser humano não vai virar obsoleto(pelo menos não por enquanto) ao mesmo tempo que aprimorei meu uso de workflow de multiagentes para fazer mais com menos custo.

<Relato individual de Alexandre Moreno>

Durante o desenvolvimento do projeto TASI, participei principalmente da integração com Modelos de Linguagem, da construção da camada de testes automatizados e da reorganização do repositório em uma estrutura de módulos mais clara. Minha atuação buscou garantir que a aplicação tivesse um núcleo de análise robusto, com decisões técnicas documentadas e cobertas por testes, e que a base de código refletisse a evolução da proposta ao longo das entregas.

No repositório, minha principal contribuição técnica foi a implementação do LLMIntegrationService, responsável por integrar a aplicação a dois provedores de LLM — Groq e Gemini — e por decidir dinamicamente para qual deles cada auditoria deve ser roteada. Essa lógica de roteamento foi consolidada na ADR-005 e considera arquivos sensíveis no diff (autenticação, secrets, infraestrutura, CI/CD) e o tamanho total do diff, de forma a equilibrar custo, latência e capacidade de contexto. Também escrevi o contrato AnalysisRecord, que padronizou a saída da análise, o módulo inicial de prompts em utils/prompts.ts e o módulo de configuração de ambiente. Complementarmente, precisei ajustar o modelo Groq quando o mixtral-8x7b-32768 foi descontinuado pela plataforma, o que reforçou a necessidade de tratar dependências de LLM gratuitas como pontos de instabilidade previsíveis, e não como infraestrutura confiável.

Na parte de qualidade, montei a suite inicial de testes unitários e de integração usando Vitest, cobrindo o roteamento entre provedores, o parsing de resposta da LLM, a validação do contrato AnalysisRecord, a montagem do prompt e a integração com as APIs externas. Além disso, conduzi uma refatoração ampla do repositório, movendo os artefatos de documentação para uma pasta docs/ dedicada e isolando a POC do extrator de PR em apps/poc-diff-reader/, o que preparou a base pra promoção do extrator ao GitHubExtractorService na fase seguinte. Essa combinação — decisão de roteamento explícita, contrato compartilhado e cobertura de testes — foi o que permitiu que os demais integrantes evoluíssem os comandos da CLI, a persistência e o gerador de relatório sobre uma fundação previsível.

<Relato individual de Stela Nascimento —>

Durante o desenvolvimento do projeto TASI, participei de atividades relacionadas à definição do escopo, organização dos artefatos, documentação técnica, implementação de funcionalidades no repositório e análise de segurança da solução. Minha atuação contribuiu tanto para a construção da aplicação quanto para o registro do processo de desenvolvimento assistido por IA, conforme proposto pela disciplina.

No início do projeto, colaborei com a definição do MVP e com a organização dos requisitos funcionais e não funcionais. Essa etapa foi importante para delimitar o que seria desenvolvido, quais funcionalidades seriam priorizadas e quais critérios de qualidade deveriam ser considerados na solução. Também participei da estruturação de documentos em Markdown, como o escopo do MVP, canvas de experimento, requisitos e registros de decisões arquiteturais.

Além disso, contribuí para a documentação da arquitetura do projeto utilizando o modelo C4, ajudando a representar a aplicação, seus principais componentes e responsabilidades. Também participei da elaboração do Catálogo de Prompts, registrando os prompts utilizados pelo grupo para apoiar tarefas como análise, geração de código, documentação e revisão. Esse catálogo foi importante para evidenciar como a equipe utilizou Inteligência Artificial ao longo do processo de desenvolvimento.

No repositório, minha principal contribuição técnica foi a implementação do gerador de relatório em Markdown. Essa funcionalidade organiza os resultados de uma análise em seções como status, criticidade, mudanças identificadas, gaps, justificativa e recomendação. Com isso, a aplicação passou a gerar uma saída mais clara, estruturada e fácil de interpretar.

Também participei da construção do relatório de segurança do Projeto TASI, analisando os principais riscos relacionados ao uso da aplicação em um ambiente de CI/CD. O foco dessa análise foi entender como a ferramenta manipula dados sensíveis, como código privado, diffs de Pull Requests, secrets, relatórios gerados e integrações com provedores externos de LLM. A partir disso, considerei aspectos de Application Security, governança e auditoria para identificar pontos críticos da solução.

Entre os pontos analisados, estiveram a necessidade de mascaramento de dados antes do envio para a LLM, o bloqueio automático de Pull Requests com criticidade crítica, o controle adequado de permissões no GitHub Actions e a proteção do histórico de análises armazenado no MongoDB. Essa etapa foi importante para compreender que uma ferramenta automatizada de auditoria precisa ser pensada não apenas pelo lado funcional, mas também pela perspectiva de segurança, rastreabilidade, minimização de dados e conformidade com boas práticas, como LGPD e ISO 27001.

Também acompanhei a validação da aplicação e a organização geral da documentação, garantindo que os artefatos produzidos estivessem coerentes com a proposta do projeto e com as entregas solicitadas. De forma geral, minha participação ajudou a conectar a parte técnica da aplicação com a documentação do processo e com a análise de segurança, contribuindo para que o projeto TASI fosse entregue de maneira mais estruturada, compreensível e alinhada aos objetivos da disciplina.

<Relato individual de Reilson Fonseca — preencher à mão>

Durante o desenvolvimento do projeto TASI, minha participação abrangeu desde a definição estratégica do produto até a implementação técnica de componentes essenciais da aplicação.

Na fase de Exposição, contribuí com a criação das duas personas que guiaram as decisões do projeto ao longo de todos os movimentos: Carolina Oliveira, PMO Sênior sem expertise técnica que dependia de desenvolvedores para validar documentação, e Rafael Santos, Desenvolvedor Full Stack que sofria com o esquecimento de atualizar docs e as interrupções constantes durante revisões. Definir essas personas foi importante porque trouxe clareza sobre para quem estávamos construindo a ferramenta e quais dores precisávamos resolver de fato.

Na parte de persistência, implementei o [AnalysisRepository](#), o componente responsável por salvar e recuperar o histórico de auditorias no MongoDB. Além das operações básicas de CRUD, criei índices por data de criação e por repositório para garantir que as consultas mais frequentes — como buscar as auditorias mais recentes de um repositório específico — fossem eficientes sem precisar varrer toda a coleção.

Na frente de qualidade, escrevi os testes automatizados de roteamento ([routing.test.ts](#)), que validam a lógica central do ADR-005: garantir que o sistema use o Groq no caminho comum e acione o Gemini automaticamente nos casos que exigem mais robustez, como arquivos de autenticação, configurações de ambiente, infraestrutura e diffs muito grandes. Esses testes foram essenciais para assegurar que uma das decisões arquiteturais mais críticas do projeto não quebrasse silenciosamente durante a evolução do código.

A disciplina me proporcionou uma visão mais madura sobre o desenvolvimento assistido por IA. Aprendi que a IA é uma ferramenta poderosa de aceleração, mas que o julgamento humano continua indispensável — seja para validar o que foi gerado, para tomar decisões de arquitetura ou para garantir que o produto final realmente resolve o problema de quem vai usá-lo.

11. Apêndices

Os apêndices reúnem os artefatos produzidos ao longo do projeto.

- Workflow Document completo: docs/workflow.MD (diário de bordo do desenvolvimento assistido por IA, com as três camadas de economicidade detalhadas por fase).
- 14 canvases/artefatos na íntegra: Canvas de Estratégia e Ação, Personas, Missão/Visão, Métricas de Sucesso, Matriz Impacto×Esforço, Escopo MVP, C4 Model, Catálogo de Prompts, Canvas de Experimento, ADRs, Canvas de Testes/Validação, Checklist de Lançamento, Painel de Feedback/Insights e Canvas de Escalabilidade.
- Catálogo de Prompts completo: docs/CATALOGO_PROMPTS-v2.md.
- Relatório de segurança: docs/RELATORIO_SEGURANCA_APPSEC.md.
- Prints do terminal, logs de commits e quadro de tarefas: <anexar antes da entrega final>.